# Mini-projet DevSecOps

Transformation de PanierLocal – Module INFAL245 – Sujet étudiant

## Présentation

Vous êtes une équipe de consultants DevSecOps mandatée par PanierLocal, une PME qui livre ses déploiements à la main et doit passer un audit sécurité client dans 9 mois. Votre mission : poser le diagnostic, concevoir l'organisation et le pipeline cible, en construire une première version qui tourne, puis défendre une roadmap de transition devant la direction.

| Élément | Modalité |
|---|---|
| Groupes | 3 à 4 personnes |
| Durée | 1 journée |
| Rendu | Dossier court + dépôt Git + soutenance de 15 min |
| Outils | GitHub Actions, Docker, Docker Compose, le panorama des outils DevSecOps |

Le mini-projet reprend les quatre objectifs d'apprentissage du module INFAL245 :

- identifier les outils du cycle de vie DevSecOps (partie B) ;
- définir les rôles dans l'équipe (partie A) ;
- connaître les étapes de la transformation (parties A et D) ;
- concevoir le processus de livraison et de déploiement continu (parties B et C).

**Règle du jeu :** chaque choix (outil, rôle, indicateur, jalon) doit être justifié par une contrainte du contexte. Un choix sans justification ne rapporte pas de points.

## Contexte : PanierLocal

PanierLocal (SAS, 45 salariés) vend en ligne des paniers de produits de producteurs locaux, livrés chaque semaine à 6 000 abonnés particuliers. L'entreprise vient de signer un contrat avec Groupe Santé Atlantique, un groupe hospitalier qui veut approvisionner ses 12 restaurants du personnel via la plateforme.

### L'application

- Un front web (React) et une API REST (Node.js) avec une base PostgreSQL.
- Fonctions principales : catalogue des paniers, abonnements, commandes, paiement via un prestataire externe, espace producteurs.
- Code hébergé sur GitHub, un seul dépôt, une seule branche `main`.
- Production : un serveur virtuel unique chez un hébergeur cloud, qui fait tourner Nginx, l'API et la base.

### L'équipe IT actuelle

| Personne | Poste | Ce qu'il faut savoir |
|---|---|---|
| Sophie | Directrice technique (CTO) | Porte le projet, veut des résultats visibles avant l'audit |
| Karim | Développeur senior | Connaît tout le code, refait seul les mises en production |
| Léa | Développeuse | Curieuse, s'intéresse à la sécurité |
| Thomas | Développeur | Réticent : « on a toujours fait comme ça » |
| Inès | Alternante développeuse | Utilise beaucoup un assistant IA pour coder |
| Marc | Administrateur systèmes (mi-temps) | Seul à avoir les accès SSH à la production |
| Julie | Responsable commerciale | Interlocutrice du client hospitalier |

Aucun profil sécurité dans l'entreprise. Le budget annuel accordé à l'outillage est de 6 000 € (licences, SaaS et hébergement des outils compris).

## Comment on livre aujourd'hui

1. Les développeurs poussent directement sur `main`, sans revue de code.
2. Une fois par mois, le vendredi soir, Karim se connecte en SSH au serveur, fait un `git pull`, installe les dépendances et lance les migrations de base à la main.
3. Marc redémarre les services et vérifie « que le site répond ».
4. En cas de problème, on corrige directement sur le serveur.

## Ce qui s'est passé ces 12 derniers mois

| Date | Incident |
|---|---|
| Septembre 2026 | Le scan de sécurité demandé par le client révèle une bibliothèque avec une CVE critique, non mise à jour depuis 2 ans |
| Juin 2026 | Inès ajoute un paquet npm suggéré par son assistant IA ; il n'existait pas officiellement et avait été enregistré par un tiers |
| Mars 2026 | Une clé d'accès cloud est poussée dans le dépôt ; elle est utilisée pour miner de la cryptomonnaie, facture de 3 800 € |
| Décembre 2025 | Le site tombe pendant 2 jours lors du pic de commandes de Noël, après une mise en production du vendredi |

**Chiffres des 12 derniers mois :** 12 mises en production, dont 4 ont nécessité un correctif en urgence. Délai moyen entre la fin d'un développement et sa mise en production : 3 semaines. Rétablissement après incident : de quelques heures à 2 jours.

## Les demandes

La direction (Sophie) veut :

- livrer au moins une fois par semaine d'ici 6 mois, sans mises en production le vendredi soir ;
- ne plus dépendre d'une seule personne pour livrer ;
- réduire le nombre de livraisons qui cassent la production.

Le client Groupe Santé Atlantique, soumis à NIS2, impose dans le contrat (audit dans 9 mois) :

- un inventaire des composants logiciels livrés (SBOM) ;
- un processus de gestion des vulnérabilités avec délais de correction (critique < 7 jours) ;
- la traçabilité de ce qui est déployé en production ;
- la notification de tout incident de sécurité sous 24 h ;
- des environnements de recette et de production séparés.

**Contrainte forte :** l'activité ne doit pas s'arrêter pendant la transformation, et l'équipe continue de livrer des fonctionnalités.

# Partie A – Diagnostic et organisation

Objectif : montrer que vous avez compris où en est PanierLocal et qui fera quoi demain. On ne parle pas encore d'outils.

## A1. État des lieux

1. Listez les irritants du contexte et, pour chacun, sa cause racine
2. Évaluez la maturité actuelle sur 5 axes: gestion du code et revue, intégration continue, déploiement, sécurité, observabilité.
3. Reliez chaque exigence du client à ce qui manque aujourd'hui pour y répondre.

## A2. Constitution de l'équipe

1. Proposez l'organisation cible : quels rôles du cours (DevOps Engineer, Security Champion, Platform Engineer, AppSec, SRE, Product Owner…) sont tenus par qui ? Faut-il recruter, faire appel à un prestataire, ou mutualiser ?
2. Justifiez la place du client hospitalier : qui lui parle, à quel rythme, sur quels sujets (incidents, vulnérabilités, livraisons) ?
3. Traitez explicitement les cas de Karim et Marc (personnes indispensables), de Thomas (résistance) et d'Inès (code généré par IA).

## A3. Montée en compétence

Pour chaque rôle, identifiez les compétences manquantes et le moyen de les acquérir : binômage, mentorat, formation, pratiques communes (revue de code, post-mortems), veille. Restez réaliste : l'équipe continue de livrer pendant ce temps.

## A4. Outils de gestion de l'équipe

Choisissez les outils de pilotage (tickets, tableau de suivi, communication, documentation, partage avec le client) et dites ce que le client peut voir.

# Partie B – Conception du pipeline DevSecOps

Objectif : dessiner le chemin complet d'une modification, du poste du développeur jusqu'à la production surveillée, avec les contrôles de sécurité au bon endroit.

## B1. Schéma du pipeline cible

Schématisez le pipeline en reprenant les 7 étapes du panorama (poste dev, pull request, build, registry, déploiement, staging, run). Pour chaque étape, indiquez :

- les contrôles effectués et leur outil ;
- le comportement en cas d'échec : bloquant ou informatif ;
- qui est notifié.

## B2. Choix des outils

Remplissez un tableau d'outillage : une ligne par catégorie (secrets, SAST, SCA, scan d'image, SBOM, signature, DAST, observabilité, centralisation des résultats), avec l'outil retenu, l'alternative écartée et la raison.

Contraintes à respecter :

- budget outillage de 6 000 € par an ;
- GitHub Team sans Advanced Security ;
- pas de Kubernetes pour l'instant : la cible reste des conteneurs Docker sur serveurs ;
- une équipe de 5 développeurs qui ne doit pas crouler sous les alertes.

### B3. Stratégie de livraison

1. Précisez si PanierLocal doit viser la livraison continue ou le déploiement continu, et pourquoi.
2. Expliquez comment vous appliquez le principe « Build once, Scan, Promote » : où l'image est construite, comment elle est identifiée, comment elle passe de la recette à la production.
3. Décrivez la gestion des secrets (fin des clés dans le dépôt) et la gestion de la configuration par environnement.
4. Indiquez comment vous répondez à chacune des 5 exigences du client grâce au pipeline.

## Partie C – Mise en pratique

Objectif : construire une première version du pipeline sur une version simplifiée de l'API PanierLocal. Il s'agit d'une ébauche qui tourne, pas d'un pipeline parfait.

### Point de départ

Le dépôt `panierlocal-api` : une mini-API « catalogue de paniers » en Node.js avec :

- les routes `GET /health`, `GET /paniers` et `GET /paniers/:id` ;
- 7 tests unitaires (Jest) ;
- un `Dockerfile` multi-étapes avec utilisateur non root ;
- un fichier `docker-compose.yml` avec deux profils, `preprod` et `prod`, qui lancent la même image (variable `IMAGE`) avec une configuration différente ;
- un script `scripts/smoke-test.sh` pour les tests de fumée.

Commencez par lancer `npm install` puis committez le `package-lock.json` généré : la CI et le `Dockerfile` en ont besoin.

### Travail demandé

Créez le workflow `.github/workflows/ci-cd.yml` (ou `.gitlab-ci.yml`), déclenché à chaque push et à chaque pull request, avec les stages suivants.

| Stage | Contenu minimum | Bonus |
|---|---|---|
| Commit | Checkout, installation, tests unitaires, scan des secrets (Gitleaks), scan des dépendances (Trivy fs ou OSV-Scanner) | Lint du Dockerfile (Hadolint), SAST (Semgrep), rapport publié en artefact |
| Build | Construction de l'image, tag avec le SHA du commit, scan de l'image (Trivy), publication sur ghcr.io uniquement si le scan passe | SBOM (Syft ou Trivy) attaché à l'image, signature Cosign |
| Préprod | Lancement avec le profil `preprod`, test de fumée par `curl` sur `/health` | DAST ZAP baseline sur l'application lancée |
| Prod | Approbation manuelle par un reviewer (environnement GitHub protégé), lancement avec le profil `prod` de la même image, test de fumée | Vérification de la signature avant le lancement, notification de fin de déploiement |

Règles :

- l'image déployée en préprod et en prod est la même (même digest) ; aucun rebuild entre les deux ;
- aucun secret en clair dans le dépôt ni dans le workflow (utiliser les secrets GitHub) ;
- les scans démarrent en mode informatif si nécessaire, mais au moins un contrôle doit être bloquant (vulnérabilités critiques) ;
- les stages Préprod et Prod ne s'exécutent que sur `main`.

### À démontrer en fin de partie

1. Un push qui passe tout le pipeline jusqu'à l'attente d'approbation, puis la mise en production.
2. Un push volontairement fautif (test cassé, faux secret ou dépendance vulnérable) qui est arrêté au bon stage.
3. La preuve que l'image en production est celle qui a été testée (digest visible dans les logs).

## Partie D – Indicateurs et roadmap de transition

Objectif : convaincre la direction que la transformation est pilotable et réaliste en 9 mois.

### D1. Indicateurs clés

1. Calculez la situation de départ de PanierLocal sur les métriques DORA à partir des chiffres du contexte (fréquence de déploiement, délai d'exécution, taux de défaillance, délai de rétablissement) et situez-la dans les niveaux du cours.
2. Fixez une cible réaliste à 6 et 9 mois pour chacune.
3. Ajoutez au moins 3 indicateurs de sécurité et 1 indicateur culturel, en précisant comment ils seront mesurés et par quel outil.
4. Expliquez comment vous évitez que ces indicateurs deviennent des objectifs individuels.

### D2. Roadmap de transition

Construisez une roadmap en phases sur 9 mois, jusqu'à l'audit client. Pour chaque phase : objectif, actions, rôles impliqués, et critère de passage à la phase suivante. La roadmap doit tenir compte du fait que l'équipe continue de livrer.

### D3. Risques d'échec

Identifiez les 5 principaux risques d'échec de la transformation (en vous appuyant sur les anti-patterns du cours) et la parade prévue pour chacun.

## Livrables et évaluation

1. Un dossier de 6 pages maximum : diagnostic, organisation, schéma du pipeline, tableau d'outillage, indicateurs, roadmap, risques.
2. Le lien vers le dépôt Git contenant l'application et le workflow fonctionnel.